

```

from __future__ import annotations

import importlib
import logging
import os
from logging.handlers import RotatingFileHandler
from collections import defaultdict
from contextlib import asynccontextmanager
from typing import Any, Dict, Iterable, List, Tuple

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.routing import APIRoute
from fastapi.staticfiles import StaticFiles

from app.api.router_registry import RouterSpec, iter_router_specs
from app.config import settings
from app.services.operator_auth_service import
ensure_operator_auth_schema
from app.services.quote_review_service import ensure_quote_pack_schema
from app.services.system_state_service import get_system_state

logger = logging.getLogger(__name__)

def _configure_logging() -> None:
    root = logging.getLogger()
    if getattr(root, "_lmcp_logging_configured", False):
        return

    log_dir = settings.log_dir
    log_dir.mkdir(parents=True, exist_ok=True)
    formatter = logging.Formatter("%(asctime)s %(levelname)s [%(name)s]
%(message)s")

    console = logging.StreamHandler()
    console.setLevel(logging.INFO)
    console.setFormatter(formatter)

    app_log = RotatingFileHandler(log_dir / "app.log", maxBytes=5 * 1024
* 1024, backupCount=5, encoding="utf-8")
    app_log.setLevel(logging.INFO)
    app_log.setFormatter(formatter)

    error_log = RotatingFileHandler(log_dir / "error.log", maxBytes=5 *
1024 * 1024, backupCount=10, encoding="utf-8")
    error_log.setLevel(logging.ERROR)
    error_log.setFormatter(formatter)

    root.setLevel(logging.INFO)
    root.addHandler(console)
    root.addHandler(app_log)
    root.addHandler(error_log)
    root._lmcp_logging_configured = True # type: ignore[attr-defined]

_configure_logging()

```

```

def _load_router(spec: RouterSpec) -> Any:
    module = importlib.import_module(spec.module_path)
    return getattr(module, spec.attribute_name)

def _route_signatures(app: FastAPI) -> Dict[Tuple[str, str], List[str]]:
    signatures: Dict[Tuple[str, str], List[str]] = defaultdict(list)
    for route in app.routes:
        if not isinstance(route, APIRoute):
            continue
        for method in sorted(route.methods or []):
            if method in {"HEAD", "OPTIONS"}:
                continue
            signatures[(method, route.path)].append(route.name)
    return signatures

def _find_duplicate_routes(app: FastAPI) -> List[Dict[str, Any]]:
    duplicates: List[Dict[str, Any]] = []
    for (method, path), names in sorted(_route_signatures(app).items()):
        if len(names) > 1:
            duplicates.append({"method": method, "path": path,
"route_names": names})
    return duplicates

def _include_registered_routers(app: FastAPI, specs:
Iterable[RouterSpec]) -> Dict[str, Any]:
    loaded: List[Dict[str, Any]] = []
    failures: List[Dict[str, str]] = []
    seen_modules: set[str] = set()

    for spec in specs:
        if spec.module_path in seen_modules:
            continue
        seen_modules.add(spec.module_path)
        try:
            router = _load_router(spec)
            app.include_router(router)
            loaded.append(
                {
                    "name": spec.name,
                    "module": spec.module_path,
                    "prefix": getattr(router, "prefix", "") or "",
                    "tags": list(getattr(router, "tags", []) or []),
                    "route_count": len(getattr(router, "routes", []) or
[],
                )
            )
        except Exception as exc:
            failures.append({"name": spec.name, "module":
spec.module_path, "error": str(exc)})

    duplicates = _find_duplicate_routes(app)
    return {"loaded": loaded, "failures": failures, "duplicates":
duplicates}

```

```

def _allow_degraded_startup() -> bool:
    return str(os.getenv("LMCP_ALLOW_DEGRADED_STARTUP",
"false")).strip().lower() == "true"

@asynccontextmanager
async def lifespan(app: FastAPI):
    settings.ensure_directories()
    app.state.database_startup_degraded = False
    app.state.database_startup_error = ""
    ensure_operator_auth_schema()
    try:
        ensure_quote_pack_schema()
    except Exception as exc:
        if not _allow_degraded_startup():
            raise
        app.state.database_startup_degraded = True
        app.state.database_startup_error = str(exc)
        logger.warning(
            "Quote pack schema initialization failed during degraded
startup; continuing without database access: %s",
            exc,
        )
    report = app.state.router_report
    if app.state.database_startup_degraded:
        logger.warning("LMCP AutoQuote API startup complete in degraded
mode.")
    else:
        logger.info("LMCP AutoQuote API startup complete")
        logger.info("Loaded routers: %s", [item["name"] for item in
report["loaded"]])
        if report["failures"]:
            logger.error("Router load failures: %s", report["failures"])
            raise RuntimeError(f"Router load failures detected:
{report['failures']}")
        if report["duplicates"]:
            logger.error("Duplicate routes detected: %s",
report["duplicates"])
            raise RuntimeError(f"Duplicate routes detected:
{report['duplicates']}")
        logger.info("Runtime static path: %s", settings.runtime_dir)
        logger.info("Monthly quotes static path: %s",
settings.monthly_quotes_dir)
        logger.info("Submission proofs static path: %s",
settings.submission_proofs_dir)
        logger.info("Portal submission static path: %s",
settings.portal_submission_dir)
        logger.info("Final submission static path: %s",
settings.final_submission_dir)
        logger.info("Proof center static path: %s",
settings.proof_center_dir)
        yield
        logger.info("LMCP AutoQuote API shutdown complete")

def build_application() -> FastAPI:

```

```
app = FastAPI(title=settings.app_name, version=settings.app_version,
lifespan=lifespan)
```

```
allow_origins = list(settings.cors_origins) if settings.cors_origins
else ["*"]
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=allow_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
mounts = (
    ("/downloads", settings.monthly_quotes_dir, "downloads"),
    ("/runtime", settings.runtime_dir, "runtime"),
    ("/proofs", settings.submission_proofs_dir, "proofs"),
    ("/portal-runtime", settings.portal_submission_dir, "portal-
runtime"),
    ("/final-submission-runtime", settings.final_submission_dir,
"final-submission-runtime"),
    ("/proof-center-runtime", settings.proof_center_dir, "proof-
center-runtime"),
    ("/handwriting-runtime", settings.handwriting_runtime_dir,
"handwriting-runtime"),
    ("/tender-form-runtime", settings.tender_form_runtime_dir,
"tender-form-runtime"),
    ("/clickable-navigation-v40-runtime",
settings.clickable_navigation_runtime_dir, "clickable-navigation-v40-
runtime"),
)
for mount_path, directory, name in mounts:
    app.mount(mount_path, StaticFiles(directory=str(directory)),
name=name)
```

```
app.state.router_report = _include_registered_routers(app,
iter_router_specs())
return app
```

```
app = build_application()
```

```
def _base_status_payload() -> Dict[str, Any]:
    report = app.state.router_report
    return {
        "version": settings.app_version,
        "environment": settings.environment,
        "loaded_routers": [item["name"] for item in report["loaded"]],
        "loaded_routers_count": len(report["loaded"]),
        "failed_routers": report["failures"],
        "failed_routers_count": len(report["failures"]),
        "duplicate_routes": report["duplicates"],
        "duplicate_routes_count": len(report["duplicates"]),
        "downloads_url": "/downloads",
        "runtime_url": "/runtime",
        "portal_runtime_url": "/portal-runtime",
        "final_submission_runtime_url": "/final-submission-runtime",
```

```

    "proof_center_runtime_url": "/proof-center-runtime",
    "submission_proofs_url": "/proofs",
    "business_rules": {
        "country": settings.target_country,
        "focus": "supply and delivery tenders only",
        "excluded_segments": list(settings.excluded_tender_segments),
        "minimum_profit_margin_zar":
settings.minimum_profit_margin_zar,
        "minimum_supply_margin_ratio":
settings.minimum_supply_margin_ratio,
        "final_submit_manual_only":
settings.final_submit_manual_only,
        "require_buyer_pricing_schedule_completion":
settings.require_buyer_pricing_schedule_completion,
    },
}

```

```

def _workflow_health_payload() -> Dict[str, Any]:
    report = app.state.router_report
    loaded_routes = {
        route.path
        for route in app.routes
        if isinstance(route, APIRoute)
    }
    system_state = get_system_state()

    return {
        "status": "healthy",
        "service": settings.app_name,
        "controlled_status": system_state.get("effective_system_status",
"off"),
        "system_on": bool(system_state.get("system_on", False)),
        "emergency_stop": bool(system_state.get("emergency_stop",
False)),
        "workflow_checks": {
            "system_control_status": "/system/control/status" in
loaded_routes,
            "system_control_effective_status":
"/system/control/effective-status" in loaded_routes,
            "tender_pipeline_health": "/tender-pipeline/health" in
loaded_routes,
            "tender_pipeline_run": "/tender-pipeline/run" in
loaded_routes,
            "quote_pack_surface": any(
                path.startswith("/quote-pack") or
path.startswith("/quote")
                for path in loaded_routes
            ),
            "pricing_surface": any(
                path.startswith("/pricing") or path.startswith("/quote-
compilation")
                for path in loaded_routes
            ),
            "submission_surface": any(
                path.startswith("/submission") or
path.startswith("/final-submission")
                for path in loaded_routes
            )
        }
    }

```

```

    ),
},
"router_health": {
    "loaded_routers_count": len(report["loaded"]),
    "failed_routers_count": len(report["failures"]),
    "duplicate_routes_count": len(report["duplicates"]),
},
}

```

```

@app.get("/")
def root() -> Dict[str, Any]:
    return {
        "message": f"{settings.app_name} API is running",
        "status": "ok",
        "docs_url": "/docs",
        "openapi_url": "/openapi.json",
        **_base_status_payload(),
    }

```

```

@app.get("/health")
def health() -> Dict[str, Any]:
    return {
        "status": "healthy",
        "service": settings.app_name,
        "environment": settings.environment,
        "runtime_dir": str(settings.runtime_dir),
        "log_dir": str(settings.log_dir),
        "monthly_quotes_dir": str(settings.monthly_quotes_dir),
        "submission_proofs_dir": str(settings.submission_proofs_dir),
        "portal_submission_dir": str(settings.portal_submission_dir),
        "final_submission_dir": str(settings.final_submission_dir),
        "proof_center_dir": str(settings.proof_center_dir),
        "project_root": str(settings.project_root),
        "database_configured": bool(settings.database_url),
        **_base_status_payload(),
    }

```

```

@app.get("/health/workflows")
def workflow_health() -> Dict[str, Any]:
    return {
        "message": "workflow health fetched",
        **_workflow_health_payload(),
    }

```